Tema 7

Aplicación de estructuras de almacenamiento

Programación

Tech Hub <FP>
euroformac

ÍNDICE

Objetivos de la unidad 4
1. Introducción 5
2. Estructuras 6
3. Arrays unidimensionales y multidimensionales 10
3.1 Array unidimensional 11
3.1.1 Declaración (Python) 11
3.1.2 Declaración (Java) 12
3.2 Creación de arrays unidimensionales (Python) 13
3.2.1 Creación de arrays unidimensionales (Java) 14
3.3 Inicialización de arrays unidimensionales (Python) 16
3.3.1 Inicialización con valores predeterminados 16
3.4 Acceso a los elementos de arrays unidimensionales (Python) 18
3.4.1 Acceso a los elementos de arrays unidimensionales (Java) 19
3.4.2 Accediendo a elementos individuales 19
3.4.3 Accediendo a todos los elementos con un bucle for 20
3.4.4 Accediendo a subconjuntos de elementos con un bucle for 20
3.5 Recorridos sobre arrays unidimensionales 21
3.5.1 Phyton 21
3.5.1.1 Recorriendo todos los elementos 21
3.5.2 Java 22
3.6 Búsquedas en arrays unidimensionales 24
3.6.1 (Python) 24
3.6.2 (Java) 26
3.7 Ordenaciones en arrays unidimensionales 28
3.7.1 (Python) 28
3.7.2 (Java) 30
4. Array multidimensionales 33
4.1 Declaración en arrays multidimensionales 33
4.1.1 Python 33

4.1.2 Java 34
4.2 Creación de arrays multidimensionales 36
4.2.1 Python 36
4.2.2 Java 37
4.3 Inicialización de arrays multidimensionales 38
4.3.1 Python 38
4.3.2 Java 40
4.4 Acceso a los elementos de arrays multidimensionales 41
4.4.1 Python 41
4.4.2 Java 42
4.5 Recorridos sobre arrays multidimensionales 44
4.5.1 Python 44
4.5.2 Java 44
4.6 Búsquedas en arrays multidimensionales 46
4.6.1 Python 46
4.6.2 Java 47
4.7 Ordenaciones en arrays multidimensionales 49
4.7.1 Python 49
4.7.2 Java 50
5. Cadena de caracteres 53
5.1 Declaración (Python) 53
5.1.1 Creación de cadenas de caracteres (Python) 54
5.1.2 Inicialización (Python) 54
5.1.3 Operaciones: Acceso a elementos, conversaciones y concatenación(Python) 55
5.2 Declaración (Java) 56
5.3 Creación de cadenas de caracteres (Java) 57
5.4 Inicialización (Java) 58
5.5 Operaciones: Acceso a elementos, conversaciones, concatenación (Java) 59
6. Resumen 61
Glosario 62
Bibliografía 63

Objetivos de la unidad

Los objetivos en este módulo son los siguientes:

- Comprender los conceptos fundamentales de las estructuras de almacenamiento, como los arrays, listas, colas, pilas, conjuntos y diccionarios.
- Saber cómo implementar y utilizar las estructuras de almacenamiento en Python y Java.

Tech Hub <FP> euroformac

1. Introducción

En programación, una estructura de almacenamiento de datos es un tipo de dato que permite guardar y organizar información de forma eficiente y fácilmente accesible para su posterior uso. Las estructuras de almacenamiento de datos se utilizan para almacenar datos en programas de computadora y se clasifican en diferentes tipos, según su funcionalidad.

En esta unidad aprenderemos cómo utilizar diferentes tipos de estructuras de datos para organizar y acceder a la información de manera eficiente en tus programas. A lo largo de la unidad, exploraremos las estructuras de almacenamiento de datos más comunes, como arreglos, listas, pilas, colas, árboles, grafos y tablas de hash. Descubriremos las ventajas y desventajas de cada una de estas estructuras, y cómo elegir la más adecuada según las necesidades específicas de tu programa y los datos que deseas almacenar y procesar.

Tech Hub <FP> euroformac

2. Estructuras

En Java, existen diferentes estructuras de almacenamiento que permiten guardar y organizar datos de diferentes maneras. Estas estructuras se dividen en dos grandes categorías: arreglos (arrays) y colecciones (collections).

- **Arreglos/Arrays**: Los arreglos son estructuras de almacenamiento que permiten guardar una colección de elementos del mismo tipo de dato en una sola variable. Se puede acceder a cada elemento del arreglo mediante un índice numérico.
  Por ejemplo, si queremos almacenar una lista de 5 números enteros en un arreglo, podemos hacer lo siguiente:

```
int[] numeros = new int[5];    // Declaración array

numeros[0] = 1;                // Primera posición del array

numeros[1] = 2;                // Segunda posición del array

numeros[2] = 3;                // Tercera posición del array

numeros[3] = 4;                // Cuarta posición del array

numeros[4] = 5;                // Cuarta posición del array
```

En este ejemplo, creamos un arreglo de tamaño 5, y luego agregamos los números del 1 al 5 en cada posición del arreglo. Podemos acceder a cada uno de estos números mediante su índice, como se muestra a continuación:

```
int primerNumero = numeros[0]; // 1

int segundoNumero = numeros[1]; // 2

int tercerNumero = numeros[2]; // 3
```

- **Colecciones**: Las colecciones son estructuras de almacenamiento más flexibles que los arreglos, ya que permiten guardar elementos de diferentes tipos de datos y su tamaño puede cambiar dinámicamente. En Java, existen varias implementaciones de colecciones, como ArrayList, LinkedList, HashSet, TreeMap, entre otros.
  Por ejemplo, si queremos almacenar una lista de nombres en un ArrayList, podemos hacer lo siguiente:

```
ArrayList<String> nombres = new ArrayList<String>();

nombres.add("Juan");

nombres.add("María");

nombres.add("Pedro");
```

En este ejemplo, creamos un ArrayList llamado nombres, y agregamos tres nombres a la lista usando el método add(). Podemos acceder a cada uno de estos nombres mediante su índice, como se muestra a continuación:

```
String primerNombre = nombres.get(0); // Se estaría obteniendo el valor "Juan"

String segundoNombre = nombres.get(1); // Se estaría obteniendo el valor "María"

String tercerNombre = nombres.get(2); // Se estaría obteniendo el valor "Pedro"
```

Además, las colecciones en Java ofrecen una gran cantidad de métodos y funcionalidades que permiten manipular los elementos almacenados en ellas de diferentes formas, como ordenarlos, buscarlos, eliminarlos, entre otros.

En Python, existen diferentes estructuras de almacenamiento que permiten guardar y organizar datos de diferentes maneras. Estas estructuras se dividen en dos grandes categorías: secuencias y colecciones.

- **Secuencias**: Las secuencias son estructuras de almacenamiento que permiten guardar una colección de elementos del mismo tipo de dato en un orden específico. Las dos secuencias más comunes en Python son las listas y las tuplas.
    - **Listas:** Las listas son estructuras de almacenamiento que permiten guardar una colección de elementos del mismo tipo de dato en un orden específico. Las listas son mutables, es decir, se pueden agregar, modificar y eliminar elementos en ellas después de que se hayan creado.
      Por ejemplo, si queremos almacenar una lista de 5 números enteros en una lista, podemos hacer lo siguiente:

```
// Declaración de Lista

numeros = [1, 2, 3, 4, 5]
```

En este ejemplo, creamos una lista llamada números, y agregamos los números del 1 al 5 en la lista. Podemos acceder a cada uno de estos números mediante su índice, como se muestra a continuación:

```
primer_numero = numeros[0]  // 1

segundo_numero = numeros[1] // 2

tercer_numero = numeros[2]  // 3
```

- **Tuplas:** Las tuplas son estructuras de almacenamiento similares a las listas, pero son inmutables, es decir, no se pueden modificar una vez que se han creado. Una forma de crear tuplas es utilizando paréntesis en vez de corchetes.
  Por ejemplo, si queremos almacenar una tupla de 3 nombres, podemos hacer lo siguiente:

```
// Declaración de tuplas

nombres = ("Juan", "María", "Pedro")
```

En este ejemplo, creamos una tupla llamada nombres, y agregamos tres nombres a la tupla. Podemos acceder a cada uno de estos nombres mediante su índice, como se muestra a continuación:

```
primer_nombre = nombres[0]      // "Juan"

segundo_nombre = nombres[1]     // "María"

tercer_nombre = nombres[2]      // "Pedro"
```

- **Colecciones**: Las colecciones son estructuras de almacenamiento más flexibles que las secuencias, ya que permiten guardar elementos de diferentes tipos de datos y su tamaño puede cambiar dinámicamente. En Python, existen varias implementaciones de colecciones, como los diccionarios y los conjuntos.
  - **Diccionarios:** Los diccionarios son estructuras de almacenamiento que permiten guardar una colección de elementos de diferentes tipos de datos en un formato de clave-valor. Cada elemento en un diccionario tiene una clave única que se utiliza para acceder al valor correspondiente.
    Por ejemplo, si queremos almacenar los datos de una persona (nombre, edad, género) en un diccionario, podemos hacer lo siguiente:

```
//Ejemplo de Diccionario

persona = {"nombre": "Juan", "edad": 30, "genero": "Masculino"}
```

En este ejemplo, creamos un diccionario llamado persona, y agregamos los datos de la persona utilizando claves y valores. Podemos acceder a cada uno de estos datos mediante su clave, como se muestra a continuación:

```
nombre = persona["nombre"] # "Juan"

edad = persona["edad"] # 30
```

- **Conjuntos:** Una colección no ordenada y sin duplicados de elementos que son inmutables se denomina conjunto en programación. Se utilizan principalmente para realizar operaciones matemáticas como unión, intersección y diferencia.

```
# Ejemplo de conjuntos

mi_conjunto = {1, 2, 3, 4, 5}
```

3. Arrays unidimensionales y multidimensionales

Definición

Un **array** es una colección ordenada de elementos del mismo tipo, a los que se accede mediante un índice numérico.

En el ámbito de la programación, un array es una estructura de almacenamiento de datos que posibilita el almacenamiento de un conjunto de elementos homogéneos. Su finalidad es guardar información en programas informáticos y se categorizan en dos tipos: unidimensionales y multidimensionales.

- Un **array unidimensional**: Una estructura de datos llamada array unidimensional permite almacenar elementos en una sola dimensión y se accede a cada elemento mediante un índice numérico que indica su posición en el array. El primer elemento del array se identifica con el índice 0, y el índice se incrementa en 1 para cada elemento subsiguiente del array.
- Un **array multidimensional**: Una estructura de almacenamiento de datos en programación es el array multidimensional que permite almacenar elementos en varias dimensiones y representar datos en forma de matrices o tablas. Cada elemento del array se identifica por un conjunto de índices numéricos que indican su posición en cada dimensión, lo que facilita el acceso a los datos.

Importante

Los arrays son estructuras de datos fundamentales en la programación, utilizados para almacenar colecciones de elementos del mismo tipo.

Los arrays multidimensionales pueden tener cualquier número de dimensiones, pero en la práctica se utilizan principalmente arrays de dos y tres dimensiones. Los arrays multidimensionales se utilizan para almacenar y procesar datos en forma de tablas, matrices, imágenes, videos y otros tipos de datos multidimensionales.

Sabías que

Los arrays en Java son objetos, mientras que en Python son listas nativas del lenguaje.

3.1 Array unidimensional

Un Array unidimensional es una estructura de datos que nos permite almacenar una lista de valores del mismo tipo en una sola variable. Cada elemento del Array tiene un índice que podemos utilizar para acceder a él. A continuación, veremos cómo se declaran, crean, inicializan, el acceso a los elementos, recorridos, búsquedas y ordenaciones.

3.1.1 Declaración (Python)

En Python, no hay un tipo de datos específico para los arrays, pero podemos usar las listas para crear arrays unidimensionales. Las listas en Python son una estructura de datos muy versátil que puede contener diferentes tipos de elementos, como enteros, cadenas, flotantes, entre otros.

Para declarar un array unidimensional en Python, simplemente creamos una lista con los elementos que deseamos almacenar en el array. Por ejemplo, si queremos un array que contenga los números del 1 al 5, podemos declararlo de la siguiente manera:

```
// Declaración array unidimensional con números del 1 al 5

mi_array = [1, 2, 3, 4, 5]
```

También podemos crear un array vacío y agregar elementos después. Para hacer esto, podemos declarar la lista vacía y luego agregar elementos utilizando el método append(). Por ejemplo, si queremos un array que contenga los nombres de algunos países, podemos hacer lo siguiente:

```
//Array de paises

paises = []                 // Creamos lista vacía llamada 'paises'

paises.append("España")     // Agregamos España a la lista
```

```
paises.append("Francia")    // Agregamos Francia a la lista
paises.append("Italia")     // Agregamos Italia a la lista
paises.append("Portugal")   // Agregamos Portugal a la lista
```

En este ejemplo, primero declaramos una lista vacía llamada paises, y luego agregamos elementos usando el método append().

También podemos crear arrays de diferentes tipos de datos. Por ejemplo, si queremos un array que contenga nombres y edades, podemos hacer lo siguiente:

```
//Array con diferentes tipos de datos:
datos = ["Juan", 25, "Pedro", 30, "Maria", 27]
```

En este ejemplo, estamos creando un array que contiene cadenas de caracteres y enteros.

Recuerda que, en Python, las listas son dinámicas, lo que significa que pueden cambiar su tamaño en cualquier momento. Es posible añadir o eliminar elementos de la lista en cualquier momento, proporcionando una gran flexibilidad en su manejo.

3.1.2 Declaración (Java)

Podemos declarar una colección de elementos del mismo tipo de datos en Java mediante la creación de un array. Para declarar un array unidimensional en Java, se puede seguir la siguiente sintaxis:

```
//Declaración array unidmensional
tipoDato[] nombreArray = new tipoDato[tamaño];
```

Donde tipoDato es el tipo de dato que se almacenará en el array (por ejemplo, int, float, String, etc.), nombreArray es el nombre que se le dará al array y tamaño es el número de elementos que se almacenarán en el array.

A continuación, se presenta un ejemplo de declaración de un array unidimensional de enteros con tamaño 5:

```
//Declaración array de tamaño 5
```

```
int[] numeros = new int[5];
```

En este caso, se declara un array llamado numeros que almacenará valores enteros y tiene un tamaño de 5 elementos. Para acceder a un elemento específico del array, se utiliza el índice correspondiente entre corchetes, donde el primer elemento del array tiene el índice 0. Por ejemplo, para asignar un valor de 10 al primer elemento del array, se puede hacer lo siguiente:

```
//Asignar 10 al primer elemento
numeros[0] = 10;
```

Es importante tener en cuenta que una vez que se declara un array en Java, su tamaño no puede ser cambiado. Sin embargo, se pueden reemplazar los valores almacenados en cada elemento del array. También existen varios métodos y técnicas para trabajar con arrays en Java, como recorrer los elementos del array, buscar elementos específicos o ordenar los elementos que veremos más adelante.

3.2 Creación de arrays unidimensionales (Python)

En Python, los arrays unidimensionales se conocen como listas, que es una colección ordenada y mutable de elementos. Puedes agregar, eliminar o cambiar elementos en una lista después de que se ha creado. Para crear una lista, simplemente coloca los elementos que deseas en corchetes y sepáralos con comas. Aquí hay un ejemplo de una lista con 5 elementos:

```
//Creación de array unidimensional
mi_lista = [1, 2, 3, 4, 5]
```

Para acceder a un elemento específico en la lista, puedes usar su índice, que comienza desde cero. Por ejemplo, para obtener el segundo elemento de la lista anterior, puedes hacer esto:

```
//Acceso al segundo elemento
segundo_elemento = mi_lista[1]
//Mostrar por pantalla
print(segundo_elemento)
```

Esto imprimirá 2 en la consola, ya que el índice del segundo elemento es 1.

También puedes cambiar un elemento en la lista asignando un nuevo valor a su índice. Por ejemplo, para cambiar el tercer elemento de la lista anterior a 10, puedes hacer esto:

```
//Modificación elementos listas
mi_lista[2] = 10
print(mi_lista)
```

Esto imprimirá "[1, 2, 10, 4, 5]" en la consola.

Además de crear listas vacías y llenas, también puedes crear listas con un rango de números usando la función range(). Por ejemplo, para crear una lista con los números del 1 al 5, puedes hacer esto:

```
//Creación de lista con numeros del 1 al 5
lista_numeros = list(range(1, 6))
print(lista_numeros)
```

Esto imprimirá "[1, 2, 3, 4, 5]" en la consola.

3.2.1 Creación de arrays unidimensionales (Java)

En Java, un array es un objeto que almacena una colección de elementos del mismo tipo. Los arrays unidimensionales en Java son aquellos que contienen una sola dimensión de elementos. Para crear un array unidimensional en Java, se utiliza la siguiente sintaxis:

```
//Creación de array unidimensional
tipo[] nombreArray = new tipo[tamaño];
```

Donde "tipo" es el tipo de dato que deseas almacenar en el array (por ejemplo, int, double, String, etc.), "nombreArray" es el nombre que le das al array y "tamaño" es la cantidad de elementos que deseas que tenga el array.

Aquí hay un ejemplo de cómo crear un array unidimensional de enteros con un tamaño de 5:

```
//Creación de array unidimensional
```

```
int[] miArray = new int[5];
```

En este caso, hemos creado un array llamado "miArray" que contiene 5 elementos enteros y cuyos valores iniciales son 0.
Puedes asignar valores a los elementos del array utilizando su índice. Los índices en Java comienzan en 0. Aquí hay un ejemplo de cómo asignar valores a los elementos de "miArray":

```
//Asignar valores a elementos del array
miArray[0] = 10;
miArray[1] = 20;
miArray[2] = 30;
miArray[3] = 40;
miArray[4] = 50;
```

Ahora, "miArray" contiene los valores {10, 20, 30, 40, 50}.
Para acceder a los valores del array, también puedes utilizar su índice. Aquí hay un ejemplo de cómo imprimir el valor del segundo elemento del array:

```
//Imprimir el segundo elemento del array
System.out.println(miArray[1]);
```

Esto imprimirá "20" en la consola.
También puedes crear un array con valores iniciales utilizando la siguiente sintaxis:

```
//Creación de array unidimensional
tipo[] nombreArray = {valor1, valor2, ..., valorN};
```

Por ejemplo, aquí hay un ejemplo de cómo crear un array de cadenas con algunos valores iniciales:

```
//Creación de array unidimensional
String[] miArrayDeCadenas = {"hola", "mundo", "en", "Java"};
```

Este array contiene los valores {"hola", "mundo", "en", "Java"}.

3.3 Inicialización de arrays unidimensionales (Python)

En Python, los arrays unidimensionales se conocen como listas, que es una colección ordenada y mutable de elementos. Puedes inicializar una lista de varias maneras diferentes, ya sea con valores predeterminados o con valores generados automáticamente.

3.3.1 Inicialización con valores predeterminados

Para inicializar una lista con valores predeterminados, simplemente coloca los valores entre corchetes y sepáralos con comas. Aquí hay un ejemplo de cómo inicializar una lista con algunos valores de enteros:

```
//Ejemplo de inicialización de lista

miLista = [1, 2, 3, 4, 5]
```

También puedes inicializar una lista con valores de otros tipos de datos, como cadenas o booleanos. Por ejemplo:

```
//Ejemplo de inicialización de lista con tipos diferentes de datos

miListaDeCadenas = ["hola", "mundo", "!"]

miListaBooleana = [True, False, True]
```

- **Inicialización con valores generados automáticamente**

Python tiene una función incorporada llamada range() que te permite generar una lista de números enteros automáticamente. Puedes utilizar esta función para inicializar una lista con valores generados automáticamente. Por ejemplo, para crear una lista de enteros del 1 al 5, puedes hacer lo siguiente:

```
//Ejemplo de inicialización de lista

miListaNumerica = list(range(1, 6))
```

Esto generará una lista de enteros que contiene los valores {1, 2, 3, 4, 5}.

- **Inicialización de una lista vacía**

También puedes inicializar una lista vacía y agregar elementos posteriormente utilizando el método append(). Por ejemplo:

```
//Ejemplo de inicialización de lista

miListaVacia = []

miListaVacia.append(1)

miListaVacia.append(2)

miListaVacia.append(3)
```

Esto generará una lista que contiene los valores {1, 2, 3}.

- **Inicialización de arrays unidimensionales (Java)**

En Java, un array unidimensional es una colección ordenada de elementos del mismo tipo de datos. Puedes inicializar un array de varias maneras diferentes, ya sea con valores predeterminados o con valores generados automáticamente.

- **Inicialización con valores predeterminados**

Para inicializar un array con valores predeterminados, utiliza la siguiente sintaxis:

```
//Ejemplo de inicialización de lista
tipo[] nombreArray = {valor1, valor2, ..., valorN};
```

Aquí, "tipo" es el tipo de dato que deseas almacenar en el array (por ejemplo, int, double, String, etc.), "nombreArray" es el nombre que le das al array y "valor1, valor2, ..., valorN" son los valores que deseas almacenar en el array. Por ejemplo, para inicializar un array de enteros con algunos valores, puedes hacer lo siguiente:

```
//Ejemplo de inicialización de lista
int[] miArray = {1, 2, 3, 4, 5};
```

También puedes inicializar un array con valores de otros tipos de datos, como cadenas o booleanos. Por ejemplo:

```
//Ejemplo de inicialización de lista con tipos diferentes de datos
String[] miArrayDeCadenas = {"hola", "mundo", "en", "Java"};
boolean[] miArrayBooleano = {true, false, true};
```

- **Inicialización con valores generados automáticamente**

Java no tiene una función incorporada para generar automáticamente una secuencia de números como en Python, pero puedes utilizar un bucle for para generar una lista de números enteros automáticamente y luego inicializar el array con esos valores. Por ejemplo, para crear un array de enteros del 1 al 5, puedes hacer lo siguiente:

```
//Ejemplo de inicialización de lista con bucle
int[] miArrayNumerico = new int[5];
for (int i = 0; i < 5; i++) {
```

```
    miArrayNumerico[i] = i + 1;
```

Esto generará un array de enteros que contiene los valores {1, 2, 3, 4, 5}.

- **Inicialización de un array vacío**

También puedes inicializar un array vacío y agregar elementos posteriormente utilizando un bucle for. Por ejemplo:

```
//Ejemplo de inicialización de array vacío
int[] miArrayVacio = new int[3];
for (int i = 0; i < 3; i++) {
    miArrayVacio[i] = i + 1;
}
```

Esto generará un array de enteros que contiene los valores {1, 2, 3}.

3.4 Acceso a los elementos de arrays unidimensionales (Python)

En Python, los arrays unidimensionales se conocen como listas y se declaran utilizando corchetes [] y separando los elementos con comas. Para acceder a los elementos de una lista, se utiliza el índice del elemento dentro de los corchetes.

Por ejemplo, si tenemos una lista de números llamada "numeros" con los valores [10, 20, 30, 40], podemos acceder a cada uno de los elementos de la siguiente manera:

```
# Ejemplo acceso a los elementos de array unidimensional
numeros[0] # Accede al primer elemento, que es 10
numeros[1] # Accede al segundo elemento, que es 20
numeros[2] # Accede al tercer elemento, que es 30
numeros[3] # Accede al cuarto elemento, que es 40
```

En Python, los índices también pueden ser negativos, lo que significa que se cuenta desde el final de la lista hacia el principio. El último elemento de la lista se puede acceder con el índice -1, el penúltimo elemento con el índice -2, y así sucesivamente.

Por ejemplo, si queremos acceder al último elemento de la lista "numeros", podemos hacerlo de la siguiente manera:

```
# Ejemplo acceso a los elementos de array unidimensional
numeros[-1] # Accede al último elemento, que es 40
```

También podemos acceder a varios elementos de la lista a la vez utilizando la sintaxis de "slicing" de Python. Para acceder a un rango de elementos, se utilizan dos puntos dentro de los corchetes, indicando el índice de inicio y el índice de final (no inclusivo).
Por ejemplo, si queremos acceder a los primeros tres elementos de la lista "numeros", podemos hacerlo de la siguiente manera:

```
# Ejemplo acceso a los elementos de array unidimensional
numeros[0:3] # Accede a los elementos del índice 0 al índice 2
```

Esto devolverá una lista con los valores [10, 20, 30].

3.4.1 Acceso a los elementos de arrays unidimensionales (Java)

A continuación, se explica el acceso a los elementos de los arrays unidimensionales en Java con algunos ejemplos.

En Java, puedes acceder a los elementos de un array utilizando su índice. El índice de un array es un número entero que representa la posición de un elemento en el array. Los índices comienzan en cero y van hasta la longitud del array menos uno.

3.4.2 Accediendo a elementos individuales

Para acceder a un elemento individual en un array, utiliza la siguiente sintaxis:

```
//Ejemplo de acceso a los elementos de arrays unidimensionales
nombreArray[indice]
```

Aquí, "nombreArray" es el nombre del array y "indice" es el índice del elemento que deseas acceder. Por ejemplo, para acceder al segundo elemento en un array de enteros llamado "miArray", puedes hacer lo siguiente:

```
//Ejemplo de acceso al segundo elemento

int segundoElemento = miArray[1];
```

Esto asignará el valor del segundo elemento (índice 1) del array a la variable "segundoElemento".

3.4.3 Accediendo a todos los elementos con un bucle for

Puedes utilizar un bucle for para recorrer todos los elementos de un array y realizar alguna operación en cada elemento. Por ejemplo, para imprimir todos los elementos de un array de enteros llamado "miArray", puedes hacer lo siguiente:

```
//Ejemplo de acceso con bucle for

for (int i = 0; i < miArray.length; i++) {

    System.out.println(miArray[i]);

}
```

Esto imprimirá cada elemento en una línea separada.

3.4.4 Accediendo a subconjuntos de elementos con un bucle for

También puedes utilizar un bucle for para acceder a un subconjunto de elementos en un array. Por ejemplo, para imprimir los elementos en un array de enteros desde el segundo hasta el cuarto elemento, puedes hacer lo siguiente:

```
//Ejemplo de acceso con bucle for

for (int i = 1; i <= 3; i++) {

    System.out.println(miArray[i]);

}
```

Esto imprimirá los elementos {2, 3, 4} en una línea separada.

3.5 Recorridos sobre arrays unidimensionales

3.5.1 Phyton

De manera genérica para recorrer un array unidimensional, necesitas seguir los siguientes pasos explicados a continuación:

1. Inicializa un índice de bucle a 0.
2. Verifica si el índice es menor que el tamaño del array.
3. Si el índice es menor que el tamaño del array, accede al elemento del array utilizando el índice y haz lo que necesites hacer con ese elemento.
4. Incrementa el índice en 1.
5. Repite los pasos 2-4 hasta que el índice sea mayor o igual que el tamaño del array.

En Python, puedes recorrer un array unidimensional utilizando un bucle for y accediendo a cada elemento utilizando su índice.

3.5.1.1 Recorriendo todos los elementos

Para recorrer todos los elementos en un array unidimensional, puedes utilizar el siguiente bucle for:

```
for elemento in nombreArray:
    # hace algo con el elemento
```

Aquí, "nombreArray" es el nombre del array y "elemento" es una variable que toma el valor de cada elemento en el array en cada iteración del bucle. Por ejemplo, para imprimir todos los elementos en un array de enteros llamado "miArray", puedes hacer lo siguiente:

```
for elemento in miArray:
print(elemento)
```

Esto imprimirá cada elemento en una línea separada.

- **Recorriendo con un índice**

Si necesitas acceder a los índices de los elementos mientras recorres el array, puedes utilizar la función range() para generar una secuencia de índices. Por

ejemplo, para imprimir cada elemento junto con su índice en un array de enteros llamado "miArray", puedes hacer lo siguiente:

```
for i in range(len(miArray)):
    print("El elemento en el índice", i, "es", miArray[i])
```

Esto imprimirá cada elemento junto con su índice en una línea separada.

- **Recorriendo en orden inverso**

También puedes recorrer el array en orden inverso utilizando la función reversed(). Por ejemplo, para imprimir todos los elementos en un array de enteros llamado "miArray" en orden inverso, puedes hacer lo siguiente:

```
for elemento in reversed(miArray):
    print(elemento)
```

Esto imprimirá cada elemento en una línea separada en orden inverso.

Aquí hay un ejemplo de código en Python que muestra cómo recorrer un array unidimensional e imprimir cada uno de sus elementos:

```
# Declarar el array
numeros = [10, 20, 30, 40, 50]


# Recorrer el array e imprimir cada elemento
for i in range(len(numeros)):
    print(numeros[i])
```

En este ejemplo, la variable numeros contiene un array con 5 elementos. El bucle for recorre el array utilizando la función range para generar una secuencia de números del 0 al 4 (que son los índices válidos del array), y accede al elemento correspondiente utilizando el índice i.

3.5.2 Java

En Java, un array unidimensional es una colección de elementos del mismo tipo, organizados en una lista ordenada. Para acceder a cada elemento del

array, es necesario usar un índice que indica la posición del elemento en la lista. Los índices en Java comienzan en cero.
Para recorrer un array unidimensional, hay dos formas principales: el bucle "for" y el bucle "foreach".

- **Bucle "for"**

El bucle "for" se utiliza para recorrer un array unidimensional mediante un índice. El siguiente ejemplo muestra cómo imprimir cada elemento del array:

```
//Ejemplo de recorrido con bucle for
int[] numeros = {1, 2, 3, 4, 5};

for(int i = 0; i < numeros.length; i++) {
    System.out.println(numeros[i]);
}
```

En este ejemplo, declaramos un array de tipo "int" llamado "numeros" y lo inicializamos con cinco valores. Luego, usamos el bucle "for" para imprimir cada elemento del array, comenzando con el índice 0 y terminando con el índice "numeros.length - 1" (que es 4 en este caso).

- **Bucle "foreach"**

El bucle "foreach" se utiliza para recorrer un array unidimensional de forma más simple y elegante. El siguiente ejemplo muestra cómo imprimir cada elemento del array utilizando el bucle "foreach":

```
//Ejemplo de recorrido con bucle foreach
int[] numeros = {1, 2, 3, 4, 5};

for(int numero : numeros) {
    System.out.println(numero);
}
```

En este ejemplo, utilizamos el bucle "foreach" para recorrer el array "numeros" y asignar cada elemento a la variable "numero". Luego, imprimimos el valor de "numero" en la consola.

3.6 Búsquedas en arrays unidimensionales

3.6.1 (Python)

En Python, un array unidimensional se llama lista. Es una colección de elementos del mismo tipo, organizados en una lista ordenada. Para acceder a cada elemento de la lista, es necesario usar un índice que indica la posición del elemento en la lista. En Python, al igual que en otros lenguajes de programación, los índices empiezan desde cero.
Hay dos tipos principales de búsquedas en una lista: la búsqueda lineal y la búsqueda binaria.

- **Búsqueda Lineal**

La búsqueda lineal consiste en recorrer el array elemento por elemento hasta encontrar el valor que se está buscando. Aquí te explico cómo se hace paso a paso:
Para buscar un valor en un array unidimensional, puedes seguir los siguientes pasos:

1. Inicializa un índice de bucle a 0.
2. Verifica si el índice es menor que el tamaño del array.
3. Si el elemento del array en la posición del índice es igual al valor que se está buscando, devuelve la posición del elemento y termina la búsqueda.
4. Si no se ha encontrado el valor, incrementa el índice en 1.
5. Repite los pasos 2-4 hasta que el índice sea mayor o igual que el tamaño del array, o hasta que se encuentre el valor buscado.

El siguiente ejemplo muestra cómo buscar un número en una lista utilizando la búsqueda lineal:

```
# Ejemplo de busqueda lineal
for i in range(len(numeros)):
    if numeros[i] == busqueda:
        encontrado = True
        print("El número", busqueda, "se encuentra en el índice", i)
        break

if not encontrado:
    print("El número", busqueda, "no se encuentra en la lista")
```

En este ejemplo, declaramos una lista de números y queremos buscar el número 8. Utilizamos un bucle "for" para recorrer cada elemento de la lista y comparamos si el número es igual a nuestro objetivo de búsqueda. Si encontramos el número, imprimimos su índice y cambiamos la variable "encontrado" a "True". Si no encontramos el número, imprimimos un mensaje de que no se encontró.

- **Búsqueda Binaria**

La búsqueda binaria es un método más eficiente de buscar un elemento en una lista ordenada. Consiste en dividir repetidamente la lista por la mitad hasta encontrar el elemento deseado. El siguiente ejemplo muestra cómo buscar el número 5 en una lista ordenada de números:

```
# Ejemplo de busqueda binaria
numeros = [1, 2, 3, 4, 5]

def busqueda_binaria(lista, valor):
    izquierda = 0
    derecha = len(lista) - 1

    while izquierda <= derecha:
        medio = (izquierda + derecha) // 2
        if lista[medio] == valor:
            return medio
        elif lista[medio] < valor:
            izquierda = medio + 1
        else:
            derecha = medio - 1

    return -1

print(busqueda_binaria(numeros, 5))
```

En este ejemplo, definimos una función llamada "busqueda_binaria" que divide repetidamente la lista por la mitad hasta encontrar el valor deseado. La

variable "izquierda" indica el índice del primer elemento de la lista, la variable "derecha" indica el índice del último elemento de la lista, y la variable "medio" indica el índice del elemento que se encuentra en el medio de la lista. Si el valor deseado es menor que el valor del elemento en el medio, se descarta la mitad derecha de la lista y se sigue buscando en la mitad izquierda. Si el valor deseado es mayor que el valor del elemento en el medio, se descarta la mitad izquierda de la lista y se sigue buscando en la mitad derecha. Si se encuentra el valor deseado, se devuelve su índice en la lista. Si no se encuentra el valor, se devuelve -1.

3.6.2 (Java)

En Java, un array unidimensional es una colección de elementos del mismo tipo, organizados en una lista ordenada. Para acceder a cada elemento del array, es necesario usar un índice que indica la posición del elemento en el array. Los índices en Java también comienzan en cero.
Para buscar un elemento en un array en Java, hay dos formas principales: la búsqueda lineal y la búsqueda binaria.

- **Búsqueda lineal**

La búsqueda lineal es un método simple y directo de buscar un elemento en un array. Consiste en recorrer el array elemento por elemento hasta encontrar el elemento deseado. El siguiente ejemplo muestra cómo buscar el número 5 en un array de números:

```
//Ejemplo de búsqueda lineal en array unidimensional


int[] numeros = {1, 2, 3, 4, 5};


public static int busquedaLineal(int[] array, int valor) {
    for (int i = 0; i < array.length; i++) {
        if (array[i] == valor) {
            return i;
        }
    }
   return -1;
}
```

```
System.out.println(busquedaLineal(numeros, 5));
```

En este ejemplo, definimos un método llamado "busquedaLineal" que recorre el array de números y compara cada elemento con el valor deseado. Si encuentra el valor, devuelve su índice en el array. Si no encuentra el valor, devuelve -1.

- **Búsqueda binaria**

La búsqueda binaria es un método más eficiente de buscar un elemento en un array ordenado. Consiste en dividir repetidamente el array por la mitad hasta encontrar el elemento deseado. El siguiente ejemplo muestra cómo buscar el número 5 en un array ordenado de números:

```
//Ejemplo de búsqueda binaria en array unidimensional

int[] numeros = {1, 2, 3, 4, 5};

public static int busquedaBinaria(int[] array, int valor) {
    int izquierda = 0;
    int derecha = array.length - 1;

    while (izquierda <= derecha) {
        int medio = (izquierda + derecha) / 2;
        if (array[medio] == valor) {
            return medio;
        } else if (array[medio] < valor) {
            izquierda = medio + 1;
        } else {
            derecha = medio - 1;
        }
    }

    return -1;
```

```
}


System.out.println(busquedaBinaria(numeros, 5));
```

En este ejemplo, definimos un método llamado "busquedaBinaria" que divide repetidamente el array por la mitad hasta encontrar el valor deseado. La variable "izquierda" indica el índice del primer elemento del array, la variable "derecha" indica el índice del último elemento del array, y la variable "medio" indica el índice del elemento que se encuentra en el medio del array. Si el valor deseado es menor que el valor del elemento en el medio, se descarta la mitad derecha del array y se sigue buscando en la mitad izquierda. Si el valor deseado es mayor que el valor del elemento en el medio, se descarta la mitad izquierda del array y se sigue buscando en la mitad derecha. Si se encuentra el valor deseado, se devuelve su índice en el array. Si no se encuentra el valor, se devuelve -1.

3.7 Ordenaciones en arrays unidimensionales

3.7.1 (Python)

En Python, un array unidimensional es una colección de elementos del mismo tipo, organizados en una lista ordenada. Para acceder a cada elemento del array, es necesario usar un índice que indica la posición del elemento en la lista. Los índices en Python empiezan desde cero, lo que significa que el primer elemento de una secuencia tiene un índice de cero, el segundo elemento tiene un índice de uno, y así sucesivamente.

Para ordenar un array en Python, hay varios algoritmos de ordenación disponibles. En este caso, explicaremos dos de los algoritmos de ordenación más utilizados: la ordenación de selección y la ordenación de burbuja.

- **Ordenación de selección**

La ordenación de selección es un algoritmo simple de ordenación que consiste en buscar el elemento mínimo en la lista y moverlo al principio de la lista. Luego, se busca el siguiente elemento mínimo en la lista restante y se mueve al segundo lugar de la lista, y así sucesivamente hasta que se ordenen todos los elementos en la lista. El siguiente ejemplo muestra cómo ordenar un array de números utilizando la ordenación de selección:

```
# Ejemplo de ordenación de selección en array unidimensional
```

```
def ordenamientoSeleccion(array):
    n = len(array)
    for i in range(n):
        min_idx = i
        for j in range(i+1, n):
            if array[j] < array[min_idx]:
                min_idx = j
        array[i], array[min_idx] = array[min_idx], array[i]
    return array

numeros = [9, 3, 1, 7, 4, 6, 8, 2, 5]
print(ordenamientoSeleccion(numeros))
```

En este ejemplo, definimos una función llamada "ordenamientoSeleccion" que recorre la lista de números y busca el elemento mínimo en la lista restante utilizando un ciclo anidado. Si encuentra un elemento mínimo, lo intercambia con el elemento en la posición actual. Al final, devuelve la lista ordenada.

- **Ordenación de burbuja**

La ordenación de burbuja es otro algoritmo de ordenación simple que compara pares de elementos adyacentes y los intercambia si están en el orden incorrecto. Luego, se repite el proceso con pares de elementos adyacentes hasta que no se necesiten más intercambios. El siguiente ejemplo muestra cómo ordenar un array de números utilizando la ordenación de burbuja:

```
# Ejemplo de ordenación de burbuja en array unidimensional


def ordenamientoBurbuja(array):
    n = len(array)
    for i in range(n):
        for j in range(n-i-1):
            if array[j] > array[j+1]:
                array[j], array[j+1] = array[j+1], array[j]
```

```
    return array


numeros = [9, 3, 1, 7, 4, 6, 8, 2, 5]
print(ordenamientoBurbuja(numeros))
```

En este ejemplo, definimos una función llamada "ordenamientoBurbuja" que recorre la lista de números y compara pares de elementos adyacentes. Si un par está en el orden incorrecto, los elementos se intercambian. El proceso se repite hasta que no se necesiten más intercambios. Al final, devuelve la lista ordenada.

3.7.2 (Java)

En Java, un array unidimensional es una colección de elementos del mismo tipo, organizados en una lista ordenada. Para acceder a cada elemento del array, es necesario utilizar un índice que indica la posición del elemento en la lista. Al igual que en Python, los índices en Java también comienzan en cero.

- **Ordenación de selección**

La ordenación de selección es un algoritmo simple de ordenación que consiste en buscar el elemento mínimo en la lista y moverlo al principio de la lista. Luego, se busca el siguiente elemento mínimo en la lista restante y se mueve al segundo lugar de la lista, y así sucesivamente hasta que se ordenen todos los elementos en la lista. El siguiente ejemplo muestra cómo ordenar un array de números utilizando la ordenación de selección en Java:

```
//Ejemplo de ordenación de seleccion en array unidimensional


public static void ordenamientoSeleccion(int[] array) {
    int n = array.length;
    for (int i = 0; i < n-1; i++) {
        int min_idx = i;
        for (int j = i+1; j < n; j++) {
            if (array[j] < array[min_idx]) {
                min_idx = j;
            }
        }
```

```
        int temp = array[min_idx];
        array[min_idx] = array[i];
        array[i] = temp;
    }
}

int[] numeros = {9, 3, 1, 7, 4, 6, 8, 2, 5};
ordenamientoSeleccion(numeros);
System.out.println(Arrays.toString(numeros));
```

En este ejemplo, definimos una función llamada "ordenamientoSeleccion" que recorre la lista de números y busca el elemento mínimo en la lista restante utilizando un ciclo anidado. Si encuentra un elemento mínimo, lo intercambia con el elemento en la posición actual. Al final, la lista se ordena y se imprime utilizando la función "Arrays.toString".

- **Ordenación de burbuja**

La ordenación de burbuja es otro algoritmo de ordenación simple que compara pares de elementos adyacentes y los intercambia si están en el orden incorrecto. Luego, se repite el proceso con pares de elementos adyacentes hasta que no se necesiten más intercambios. El siguiente ejemplo muestra cómo ordenar un array de números utilizando la ordenación de burbuja en Java:

```
//Ejemplo de ordenación de burbuja en array unidimensional

public static void ordenamientoBurbuja(int[] array) {
    int n = array.length;

    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (array[j] > array[j + 1]) {
                int temp = array[j];
                array[j] = array[j + 1];
                array[j + 1] = temp;
```

```
            }
        }
    }
}
```

En este ejemplo, definimos una función llamada "ordenamientoBurbuja" que toma un array de enteros como parámetro. Dentro de la función, utilizamos dos ciclos anidados para comparar cada par de elementos adyacentes en el array. Si el primer elemento es mayor que el segundo, los intercambia. Este proceso se repite varias veces hasta que todos los elementos estén ordenados.

4. Array multidimensionales

Un array multidimensional es una estructura de datos que se utiliza para almacenar una colección de elementos en múltiples dimensiones, como una tabla de filas y columnas. Por ejemplo, una matriz es un tipo común de array multidimensional que se utiliza para almacenar datos en una estructura de filas y columnas.
En términos más generales, un array multidimensional se puede pensar como un array de arrays. Es decir, en lugar de tener un único array unidimensional, un array multidimensional tiene varios arrays unidimensionales que están interconectados entre sí.

4.1 Declaración en arrays multidimensionales

4.1.1 Python

En Python, puedes crear arrays multidimensionales utilizando la estructura de datos de lista anidada. En una lista anidada, cada elemento de la lista es a su vez otra lista, lo que permite crear una estructura de datos en forma de matriz o tabla.
Por ejemplo, para crear una matriz de 3x3, donde cada elemento es un número entero, podríamos hacerlo de la siguiente manera:

```
#Ejemplo de declaracion de array multidimensional

my_array = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

En este ejemplo, la matriz contiene tres filas y tres columnas. Cada fila se representa como una sublista dentro de la lista principal. Los elementos individuales de la matriz se acceden utilizando dos índices, uno para la fila y otro para la columna. Por ejemplo, para acceder al elemento en la fila 2 y la columna 3 de la matriz, usamos la siguiente sintaxis:

```
elemento = matriz[1][2]   # Nota que los índices empiezan en 0

print(elemento)   # Imprime 6
```

También se puede crear una matriz vacía de tamaño n x m utilizando dos ciclos anidados. Aquí hay un ejemplo de cómo crear una matriz vacía de tamaño 2x2:

```
#Ejemplo de declaracion de array multidimensional
```

```
filas = 2
columnas = 2
matriz = []

for i in range(filas):
    fila = []
    for j in range(columnas):
        fila.append(0)
    matriz.append(fila)

print(matriz)
```

En este ejemplo, creamos una lista vacía llamada "matriz" y luego utilizamos dos ciclos anidados para llenarla con ceros. El primer ciclo itera sobre el número de filas y el segundo ciclo itera sobre el número de columnas. En cada iteración del segundo ciclo, agregamos un elemento con valor cero a la sublista actual. Luego, agregamos la sublista completa a la matriz principal utilizando el método append().

4.1.2 Java

En Java, los arrays multidimensionales se pueden declarar de forma similar a los arrays unidimensionales, pero con una sintaxis ligeramente diferente. En lugar de especificar solo la longitud del array, especificamos la longitud de cada dimensión del array.
Aquí hay un ejemplo de cómo declarar una matriz en Java de tamaño 3x3:

```
//Ejemplo de declaracion de array multidimensional
int[][] matriz = new int[3][3];
```

En este ejemplo, utilizamos la palabra clave "new" para crear una nueva instancia del array. La matriz es una estructura de datos multidimensional donde la primera dimensión representa las filas y la segunda dimensión representa las columnas. Para acceder a los elementos de la matriz, se utilizan dos índices: uno para la fila y otro para la columna. Por ejemplo, para acceder

al elemento en la fila 2 y la columna 3 de la matriz, usamos la siguiente sintaxis:

```
int elemento = matriz[1][2];   // Nota que los índices empiezan en 0

System.out.println(elemento);   // Imprime 0, ya que todos los elementos
se inicializan en 0      por defecto
```

También se puede crear una matriz vacía de tamaño n x m utilizando dos ciclos anidados. Aquí hay un ejemplo de cómo crear una matriz vacía de tamaño 2x2:

```
//Ejemplo de declaracion de array multidimensional


int filas = 2;

int columnas = 2;

int[][] matriz = new int[filas][columnas];


for (int i = 0; i < filas; i++) {

    for (int j = 0; j < columnas; j++) {

        matriz[i][j] = 0;

    }

}

System.out.println(Arrays.deepToString(matriz));
```

En este ejemplo, creamos una nueva instancia del array "matriz" con la longitud especificada de filas y columnas. Luego, utilizamos dos ciclos anidados para llenarla con ceros. El primer ciclo itera sobre el número de filas y el segundo ciclo itera sobre el número de columnas. En cada iteración del segundo ciclo, establecemos el valor del elemento en la fila y columna actual en cero. Por último, utilizamos el método "Arrays.deepToString()" para imprimir la matriz completa.

4.2 Creación de arrays multidimensionales

4.2.1 Python

En Python, los arrays multidimensionales también se llaman matrices y se pueden crear utilizando la biblioteca NumPy.
Aquí hay un ejemplo de cómo crear una matriz en Python de tamaño 3x3:

```
# Ejemplo de cómo crear una matriz en Python de tamaño 3x3
import numpy as np


matriz = np.zeros((3, 3))
```

En este ejemplo, utilizamos la función "zeros()" de NumPy para crear una matriz de tamaño 3x3. La función "zeros()" crea una matriz llena de ceros con la forma especificada. La forma se especifica utilizando una tupla de dos elementos: el número de filas y el número de columnas. La matriz resultante se guarda en la variable "matriz".
Los elementos individuales de la matriz se acceden utilizando dos índices, uno para la fila y otro para la columna. Por ejemplo, para acceder al elemento en la fila 2 y la columna 3 de la matriz, usamos la siguiente sintaxis:

```
elemento = matriz[1][2]   # Nota que los índices empiezan en 0
print(elemento)   # Imprime 0, ya que todos los elementos se inicializan
en 0 por defecto
```

También se puede crear una matriz vacía de tamaño n x m utilizando dos ciclos anidados. Aquí hay un ejemplo de cómo crear una matriz vacía de tamaño 2x2:

```
# Ejemplo de cómo crear una matriz en Python de tamaño 2x2
filas = 2
columnas = 2
matriz = np.empty((filas, columnas))


for i in range(filas):
    for j in range(columnas):
        matriz[i][j] = 0


print(matriz)
```

En este ejemplo, creamos una nueva instancia del array "matriz" utilizando la función "empty()" de NumPy con la longitud especificada de filas y columnas. Luego, utilizamos dos ciclos anidados para llenarla con ceros. El primer ciclo itera sobre el número de filas y el segundo ciclo itera sobre el número de columnas. En cada iteración del segundo ciclo, establecemos el valor del elemento en la fila y columna actual en cero. Por último, imprimimos la matriz completa utilizando la función "print()".

4.2.2 Java

En Java, los arrays multidimensionales también se llaman matrices y se pueden crear de varias maneras.
Aquí hay un ejemplo de cómo crear una matriz en Java de tamaño 3x3:

```
# Ejemplo de cómo crear una matriz en Java de tamaño 3x3:

int[][] matriz = new int[3][3];
```

En este ejemplo, utilizamos la sintaxis "new int[3][3]" para crear una matriz de tamaño 3x3. En la estructura de una matriz, la dimensión que se ubica en la posición más externa representa las filas, mientras que la dimensión interna representa las columnas. La matriz resultante se guarda en la variable "matriz".
También se puede inicializar la matriz con valores iniciales utilizando la siguiente sintaxis:

```
# Ejemplo de cómo crear una matriz

int[][] matriz = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};
```

En este ejemplo, creamos una matriz de tamaño 3x3 e inicializamos los valores de cada elemento. Los elementos individuales de la matriz se acceden utilizando dos índices, uno para la fila y otro para la columna. Por ejemplo, para acceder al elemento en la fila 2 y la columna 3 de la matriz, usamos la siguiente sintaxis:

```
int elemento = matriz[1][2];   // Nota que los índices empiezan en 0

System.out.println(elemento);   // Imprime 6
```

También se puede crear una matriz vacía utilizando dos ciclos anidados. Aquí hay un ejemplo de cómo crear una matriz vacía de tamaño 2x2:

```
// Crear una matriz vacía de tamaño 2x2
int filas = 2;
int columnas = 2;
int[][] matriz = new int[filas][columnas];

for (int i = 0; i < filas; i++) {
    for (int j = 0; j < columnas; j++) {
        matriz[i][j] = 0;
    }
}

for (int i = 0; i < filas; i++) {
    for (int j = 0; j < columnas; j++) {
        System.out.print(matriz[i][j] + " ");
    }
    System.out.println();
}
```

En este ejemplo, creamos una nueva instancia del array "matriz" utilizando la sintaxis "new int[filas][columnas]". Luego, utilizamos dos ciclos anidados para llenarla con ceros. El primer ciclo itera sobre el número de filas y el segundo ciclo itera sobre el número de columnas. En cada iteración del segundo ciclo, establecemos el valor del elemento en la fila y columna actual en cero. Finalmente, imprimimos la matriz completa utilizando dos ciclos anidados y la función "System.out.print()".

4.3 Inicialización de arrays multidimensionales

4.3.1 Python

En Python, se pueden inicializar matrices (o arrays multidimensionales) de varias maneras.
Aquí hay un ejemplo de cómo inicializar una matriz de tamaño 3x3 con valores iniciales:

```
# Ejemplo de inicializar arrays multidimensionales
matriz = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

En este ejemplo, utilizamos corchetes para crear una matriz que contiene tres listas (que representan las filas) y cada lista contiene tres elementos (que representan las columnas). Los valores iniciales se especifican directamente dentro de los corchetes. La matriz resultante se guarda en la variable "matriz".
También se puede inicializar una matriz vacía y luego llenarla con valores utilizando un bucle for anidado. Aquí hay un ejemplo de cómo inicializar una matriz de tamaño 2x2 con valores iniciales de cero:

```
# Ejemplo de inicializar arrays multidimensionales
filas = 2
columnas = 2
matriz = [[0 for j in range(columnas)] for i in range(filas)]
```

En este ejemplo, utilizamos la sintaxis de comprensión de lista para crear una matriz vacía de tamaño 2x2. El bucle for anidado itera sobre el número de filas y columnas y establece cada elemento en cero utilizando la sintaxis "[0 for j in range(columnas)] ". Finalmente, la matriz resultante se guarda en la variable "matriz".
También se puede inicializar una matriz vacía y llenarla con valores utilizando la función "numpy.zeros()". Aquí hay un ejemplo de cómo inicializar una matriz de tamaño 3x3 con valores iniciales de cero:

```
# Ejemplo de inicializar arrays multidimensionales
import numpy as np


filas = 3
columnas = 3
matriz = np.zeros((filas, columnas))
```

En este ejemplo, importamos la biblioteca NumPy y utilizamos la función "numpy.zeros()" para crear una matriz de tamaño 3x3 llena de ceros. La matriz resultante se guarda en la variable "matriz".

4.3.2 Java

En Java, se pueden inicializar matrices multidimensionales de varias maneras. Aquí hay un ejemplo de cómo inicializar una matriz de tamaño 3x3 con valores iniciales:

```
// Inicialización de arrays multidimensionales
int[][] matriz = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};
```

En este ejemplo, utilizamos corchetes para crear una matriz que contiene tres matrices (que representan las filas) y cada matriz contiene tres elementos (que representan las columnas). Los valores iniciales se especifican directamente dentro de los corchetes. La matriz resultante se guarda en la variable "matriz".
También se puede inicializar una matriz vacía y luego llenarla con valores utilizando un bucle for anidado. Aquí hay un ejemplo de cómo inicializar una matriz de tamaño 2x2 con valores iniciales de cero:

```
// Ejemplo de cómo inicializar una matriz de tamaño 2x2 con valores
iniciales de cero
int[][] matriz = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};


int filas = 2;
int columnas = 2;
int[][] matriz = new int[filas][columnas];


for (int i = 0; i < filas; i++) {
    for (int j = 0; j < columnas; j++) {
        matriz[i][j] = 0;
    }
}
```

En este ejemplo, creamos una matriz vacía de tamaño 2x2 utilizando la sintaxis "new int[filas][columnas]". Luego, utilizamos un bucle for anidado para establecer cada elemento en cero. El primer bucle for itera sobre las filas, mientras que el segundo bucle for itera sobre las columnas. Cada elemento se establece utilizando la sintaxis "matriz[i][j] = 0;". Finalmente, la matriz resultante se guarda en la variable "matriz".

También se puede inicializar una matriz vacía y llenarla con valores utilizando la función "Arrays.fill()". Aquí hay un ejemplo de cómo inicializar una matriz de tamaño 3x3 con valores iniciales de cero:

```
// Ejemplo de cómo inicializar
import java.util.Arrays;

int filas = 3;
int columnas = 3;
int[][] matriz = new int[filas][columnas];

for (int i = 0; i < filas; i++) {
    Arrays.fill(matriz[i], 0);
```

En este ejemplo, importamos la clase "java.util.Arrays" y utilizamos la función "Arrays.fill()" para establecer cada elemento de la matriz en cero. El primer argumento de la función "Arrays.fill()" es la matriz que se va a llenar, y el segundo argumento es el valor que se va a utilizar para llenar la matriz. El bucle for itera sobre las filas y llama a la función "Arrays.fill()" para establecer cada fila en cero. Finalmente, la matriz resultante se guarda en la variable "matriz".

4.4 Acceso a los elementos de arrays multidimensionales

4.4.1 Python

En Python, los elementos de una matriz multidimensional se pueden acceder utilizando la misma sintaxis que en una matriz unidimensional, pero con índices adicionales para cada dimensión.
Aquí hay un ejemplo de cómo acceder a un elemento en una matriz de tamaño 3x3:

```
# Ejemplo de cómo acceder a los elementos de array multidimensionales
matriz = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]

# Acceder al elemento en la fila 2, columna 3
elemento = matriz[1][2]
```

```
print(elemento)
```

En este ejemplo, creamos una matriz de tamaño 3x3 utilizando corchetes dobles y asignamos la matriz a la variable "matriz". Luego, utilizamos corchetes dobles para acceder al elemento en la fila 2 y columna 3 de la matriz. Nota que, en Python, los índices comienzan desde cero, por lo que el índice 1 corresponde a la segunda fila y el índice 2 corresponde a la tercera columna. Finalmente, imprimimos el valor del elemento utilizando la función "print()".
También se pueden recorrer todos los elementos de una matriz multidimensional utilizando bucles anidados. Aquí hay un ejemplo de cómo imprimir todos los elementos de una matriz de tamaño 2x3:

```
# Ejemplo de cómo imprimir todos los elementos de una matriz de tamaño 2x3
matriz = [[1, 2, 3], [4, 5, 6]]


# Recorrer todos los elementos de la matriz
for i in range(len(matriz)):
    for j in range(len(matriz[i])):
        print(matriz[i][j])
```

En este ejemplo, creamos una matriz de tamaño 2x3 utilizando corchetes dobles y asignamos la matriz a la variable "matriz". Luego, utilizamos dos bucles for anidados para recorrer todos los elementos de la matriz. El primer bucle for itera sobre las filas de la matriz, mientras que el segundo bucle for itera sobre las columnas de cada fila. El valor de cada elemento se imprime utilizando la función "print()".

4.4.2 Java

En Java, los elementos de una matriz multidimensional se pueden acceder utilizando la misma sintaxis que en una matriz unidimensional, pero con índices adicionales para cada dimensión.
Aquí hay un ejemplo de cómo acceder a un elemento en una matriz de tamaño 3x3:

```
// Ejemplo de cómo acceder a los elementos de array multidimensionales
int[][] matriz = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};
```

```
// Acceder al elemento en la fila 2, columna 3
int elemento = matriz[1][2];

System.out.println(elemento);
```

En este ejemplo, creamos una matriz de tamaño 3x3 utilizando llaves dobles y asignamos la matriz a la variable "matriz". Luego, utilizamos llaves dobles para acceder al elemento en la fila 2 y columna 3 de la matriz. Nota que, en Java, los índices comienzan desde cero, por lo que el índice 1 corresponde a la segunda fila y el índice 2 corresponde a la tercera columna. Finalmente, imprimimos el valor del elemento utilizando la función "System.out.println()". También se pueden recorrer todos los elementos de una matriz multidimensional utilizando bucles anidados. Aquí hay un ejemplo de cómo imprimir todos los elementos de una matriz de tamaño 2x3:

```
// Ejemplo de cómo acceder a los elementos de array multidimensionales
int[][] matriz = {{1, 2, 3}, {4, 5, 6}};

// Recorrer todos los elementos de la matriz
for (int i = 0; i < matriz.length; i++) {
    for (int j = 0; j < matriz[i].length; j++) {
        System.out.println(matriz[i][j]);
    }
}
```

En este ejemplo, creamos una matriz de tamaño 2x3 utilizando llaves dobles y asignamos la matriz a la variable "matriz". Luego, utilizamos dos bucles for anidados para recorrer todos los elementos de la matriz. El primer bucle for itera sobre las filas de la matriz, mientras que el segundo bucle for itera sobre las columnas de cada fila. El valor de cada elemento se imprime utilizando la función "System.out.println()".

4.5 Recorridos sobre arrays multidimensionales

4.5.1 Python

Para recorrer los elementos de un array multidimensional, podemos usar un bucle for anidado. En el caso de un array bidimensional, podemos usar dos bucles for, uno para recorrer las filas y otro para recorrer las columnas.
Por ejemplo, para recorrer los elementos del array arr creado anteriormente, podemos usar el siguiente código:

```
# Recorrer los elementos del array
for i in range(len(arr)):
    for j in range(len(arr[i])):
        print(arr[i][j])
```

En este ejemplo, el bucle exterior recorre las filas del array, mientras que el bucle interior recorre las columnas de cada fila. El índice i se usa para acceder a cada fila, y el índice j se usa para acceder a cada columna dentro de la fila.
Veamos otro ejemplo de cómo recorrer un array tridimensional en Python:

```
# Crear un array de 2x2x2
arr = [[[1, 2], [3, 4]], [[5, 6], [7, 8]]]

# Recorrer los elementos del array
for i in range(len(arr)):
    for j in range(len(arr[i])):
        for k in range(len(arr[i][j])):
            print(arr[i][j][k])
```

En este caso, tenemos un array tridimensional con dos capas, dos filas y dos columnas. Para recorrer los elementos de este array, usamos tres bucles for anidados: uno para cada dimensión del array.

4.5.2 Java

En Java, podemos recorrer los elementos de los arrays multidimensionales utilizando bucles anidados. Primeramente, vamos a explicar qué son los arrays multidimensionales.

Un array multidimensional es una colección de elementos que se organizan en filas y columnas (y posiblemente más dimensiones). Cada elemento se puede identificar mediante su posición, que se especifica mediante índices. Por ejemplo, en un array bidimensional, los elementos se organizan en filas y columnas, y se identifican mediante dos índices: uno para la fila y otro para la columna.
Aquí hay un ejemplo de cómo crear un array bidimensional en Java:

```
// Crear un array de 3x3
int[][] arr = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};
```

Para recorrer los elementos de un array multidimensional, podemos usar un bucle for anidado. En el caso de un array bidimensional, podemos usar dos bucles for, uno para recorrer las filas y otro para recorrer las columnas.
Por ejemplo, para recorrer los elementos del array arr creado anteriormente, podemos usar el siguiente código:

```
// Recorrer los elementos del array
for (int i = 0; i < arr.length; i++) {
    for (int j = 0; j < arr[i].length; j++) {
        System.out.println(arr[i][j])
```

En este ejemplo, el bucle exterior recorre las filas del array, mientras que el bucle interior recorre las columnas de cada fila. El índice i se usa para acceder a cada fila, y el índice j se usa para acceder a cada columna dentro de la fila.
Veamos otro ejemplo de cómo recorrer un array tridimensional en Java:

```
// Crear un array de 2x2x2
int[][][] arr = {{{1, 2}, {3, 4}}, {{5, 6}, {7, 8}}};


// Recorrer los elementos del array
for (int i = 0; i < arr.length; i++) {
    for (int j = 0; j < arr[i].length; j++) {
        for (int k = 0; k < arr[i][j].length; k++) {
            System.out.println(arr[i][j][k]);
        }
    }
}
```

En este caso, tenemos un array tridimensional con dos capas, dos filas y dos columnas. Para recorrer los elementos de este array, usamos tres bucles for anidados: uno para cada dimensión del array.

4.6 Búsquedas en arrays multidimensionales

4.6.1 Python

En Python, podemos buscar elementos en arrays multidimensionales utilizando bucles y condicionales. Primero, permítame explicar cómo funcionan las búsquedas en arrays multidimensionales.
En un array multidimensional, los elementos se organizan en filas y columnas (y posiblemente más dimensiones). Cada elemento se puede identificar mediante su posición, que se especifica mediante índices. Por ejemplo, en un array bidimensional, los elementos se organizan en filas y columnas, y se identifican mediante dos índices: uno para la fila y otro para la columna.
Para buscar un elemento en un array multidimensional, necesitamos recorrer todos los elementos del array y verificar si el elemento que buscamos coincide con algún elemento en el array.
Aquí hay un ejemplo de cómo buscar un elemento en un array bidimensional en Python:

```
# Crear un array de 3x3
arr = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]

# Definir el elemento que queremos buscar
elemento_buscado = 5

# Recorrer los elementos del array y buscar el elemento
for i in range(len(arr)):
    for j in range(len(arr[i])):
        if arr[i][j] == elemento_buscado:
            print("El elemento se encuentra en la posición", i, j)
```

En este ejemplo, el bucle exterior recorre las filas del array, mientras que el bucle interior recorre las columnas de cada fila. El operador == se utiliza para verificar si el elemento actual es igual al elemento buscado. Si se encuentra el elemento, se imprime su posición en el array.

Veamos otro ejemplo de cómo buscar un elemento en un array tridimensional en Python:

```
# Crear un array de 2x2x2
arr = [[[1, 2], [3, 4]], [[5, 6], [7, 8]]]

# Definir el elemento que queremos buscar
elemento_buscado = 6

# Recorrer los elementos del array y buscar el elemento
for i in range(len(arr)):
    for j in range(len(arr[i])):
        for k in range(len(arr[i][j])):
            if arr[i][j][k] == elemento_buscado:
                print("El elemento se encuentra en la posición", i, j, k)
```

En este caso, tenemos un array tridimensional con dos capas, dos filas y dos columnas. Para recorrer los elementos de este array, usamos tres bucles for anidados: uno para cada dimensión del array. Al igual que en el ejemplo anterior, utilizamos el operador == para verificar si el elemento actual es igual al elemento buscado.

4.6.2 Java

En Java, podemos buscar elementos en arrays multidimensionales utilizando bucles y condicionales. Primero, permítame explicar cómo funcionan las búsquedas en arrays multidimensionales.
En un array multidimensional, los elementos se organizan en filas y columnas (y posiblemente más dimensiones). Cada elemento se puede identificar mediante su posición, que se especifica mediante índices. Por ejemplo, en un array bidimensional, los elementos se organizan en filas y columnas, y se identifican mediante dos índices: uno para la fila y otro para la columna.
Para buscar un elemento en un array multidimensional, necesitamos recorrer todos los elementos del array y verificar si el elemento que buscamos coincide con algún elemento en el array.
Aquí hay un ejemplo de cómo buscar un elemento en un array bidimensional en Java:

```
// Crear un array de 3x3
int[][] arr = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};

// Definir el elemento que queremos buscar
int elementoBuscado = 5;

// Recorrer los elementos del array y buscar el elemento
for (int i = 0; i < arr.length; i++) {
    for (int j = 0; j < arr[i].length; j++) {
        if (arr[i][j] == elementoBuscado) {
            System.out.println("El elemento se encuentra en la posición "
+ i + "," + j);
        }
    }
}
```

En este ejemplo, el bucle exterior recorre las filas del array, mientras que el bucle interior recorre las columnas de cada fila. El operador == se utiliza para verificar si el elemento actual es igual al elemento buscado. Si se encuentra el elemento, se imprime su posición en el array.
Veamos otro ejemplo de cómo buscar un elemento en un array tridimensional en Java:

```
// Crear un array de 2x2x2
int[][][] arr = {{{1, 2}, {3, 4}}, {{5, 6}, {7, 8}}};

// Definir el elemento que queremos buscar
int elementoBuscado = 6;

// Recorrer los elementos del array y buscar el elemento
for (int i = 0; i < arr.length; i++) {
    for (int j = 0; j < arr[i].length; j++) {
        for (int k = 0; k < arr[i][j].length; k++) {
            if (arr[i][j][k] == elementoBuscado) {
```

```
                System.out.println("El elemento se encuentra en la posición " + i + "," + j + "," + k);
            }
        }
    }
}
```

En este caso, tenemos un array tridimensional con dos capas, dos filas y dos columnas. Para recorrer los elementos de este array, usamos tres bucles for anidados: uno para cada dimensión del array. Al igual que en el ejemplo anterior, utilizamos el operador == para verificar si el elemento actual es igual al elemento buscado.

4.7 Ordenaciones en arrays multidimensionales

4.7.1 Python

En Python, se pueden ordenar los elementos de un array multidimensional de varias maneras diferentes. Una forma común de ordenar los elementos es utilizando la función sorted() o el método sort(). Ambos se pueden utilizar para ordenar los elementos en orden ascendente o descendente.
Aquí hay un ejemplo de cómo ordenar un array bidimensional en Python:

```
# Crear un array de 3x3
arr = [[9, 8, 7], [6, 5, 4], [3, 2, 1]]

# Ordenar los elementos en orden ascendente
for fila in arr:
    fila.sort()

# Imprimir el array ordenado
for fila in arr:
    print(fila)
```

En este ejemplo, utilizamos el método sort() para ordenar cada fila del array en orden ascendente. Luego, imprimimos el array completo para mostrar que los elementos están ordenados.
También podemos ordenar el array multidimensional utilizando la función sorted() en Python. Aquí hay un ejemplo:

```
# Crear un array de 3x3
arr = [[9, 8, 7], [6, 5, 4], [3, 2, 1]]

# Ordenar los elementos en orden descendente utilizando la función sorted()
arr_ordenado = sorted(arr, key=lambda fila: fila[0], reverse=True)

# Imprimir el array ordenado
for fila in arr_ordenado:
    print(fila)
```

En este ejemplo, utilizamos la función sorted() para ordenar el array multidimensional en orden descendente utilizando el primer elemento de cada fila como criterio de ordenación. El parámetro reverse=True se utiliza para especificar que queremos ordenar los elementos en orden descendente. Luego, imprimimos el array ordenado.

4.7.2 Java

En Java, también podemos ordenar los elementos de un array multidimensional utilizando diferentes técnicas. Una forma común de ordenar los elementos es utilizando el método Arrays.sort(). Este método se puede utilizar para ordenar los elementos de un array multidimensional en orden ascendente o descendente.
Aquí hay un ejemplo de cómo ordenar un array bidimensional en Java utilizando Arrays.sort():

```
// Crear un array de 3x3
int[][] arr = {{9, 8, 7}, {6, 5, 4}, {3, 2, 1}};

// Ordenar los elementos en orden ascendente
for (int[] fila : arr) {
```

```
    Arrays.sort(fila);
}

// Imprimir el array ordenado
for (int[] fila : arr) {
    System.out.println(Arrays.toString(fila));
}
```

En este ejemplo, utilizamos el método Arrays.sort() para ordenar cada fila del array en orden ascendente. Luego, imprimimos el array completo para mostrar que los elementos están ordenados.
También podemos ordenar el array multidimensional utilizando la clase Arrays de Java. Aquí hay un ejemplo:

```
// Crear un array de 3x3
int[][] arr = {{9, 8, 7}, {6, 5, 4}, {3, 2, 1}};

// Ordenar los elementos en orden descendente utilizando la clase Arrays
Arrays.sort(arr, new Comparator<int[]>() {
    @Override
    public int compare(int[] fila1, int[] fila2) {
        return Integer.compare(fila2[0], fila1[0]);
    }
});

// Imprimir el array ordenado
for (int[] fila : arr) {
    System.out.println(Arrays.toString(fila));
}
```

En este ejemplo, utilizamos la clase Arrays de Java para ordenar el array multidimensional en orden descendente utilizando el primer elemento de cada fila como criterio de ordenación. Para ello, utilizamos una clase anónima que implementa la interfaz Comparator. Luego, imprimimos el array ordenado.

Más info

Este video de Programación ATS, explica detalladamente qué son los arrays y cómo funcionan:
https://www.youtube.com/watch?v=kYB8IZa5AuE

5. Cadena de caracteres

Definición

Una cadena de caracteres es una secuencia de caracteres que se almacena como un solo objeto en la memoria del programa.

Las cadenas de caracteres son una de las estructuras de datos más utilizadas en programación. Una cadena de caracteres es una secuencia de caracteres, como letras, números y símbolos, que se utilizan para representar texto en un programa.
En muchos lenguajes de programación, las cadenas de caracteres se representan como un array de caracteres, lo que significa que cada elemento del array es un carácter de la cadena. Estas cadenas pueden ser manipuladas de varias maneras, como para agregar o eliminar caracteres, compararlas con otras cadenas y buscar subcadenas dentro de ellas.
Las cadenas de caracteres son muy importantes para la entrada y salida de datos en muchos programas, como la lectura de archivos de texto y la comunicación con bases de datos y sitios web.

Importante

Las cadenas de caracteres son secuencias de caracteres utilizadas para representar texto en los programas.

5.1 Declaración (Python)

Las cadenas de caracteres, también conocidas como "strings", son una secuencia de caracteres que se pueden utilizar para almacenar y manipular texto en Python.

Sabías que

En Python, las cadenas de caracteres se pueden manipular utilizando operaciones de slicing y concatenación, lo que las hace muy flexibles.

Antes de utilizar una cadena de caracteres, es necesario declararla. Esto se hace simplemente asignando un valor de cadena a una variable. Por ejemplo, para declarar una variable llamada "nombre" y asignarle el valor de cadena "Juan", se utilizaría la siguiente expresión:

```
# Ejemplo de declaración de cadenas de caracteres

nombre = "Juan"
```

5.1.1 Creación de cadenas de caracteres (Python)

Las cadenas de caracteres se pueden crear utilizando comillas simples (' '), comillas dobles (" ") o triples comillas dobles (""" """). Las comillas simples y dobles se utilizan para cadenas de una sola línea, mientras que las triples comillas dobles se utilizan para cadenas de varias líneas. Por ejemplo, para crear una cadena de caracteres utilizando comillas simples, se utilizaría la siguiente expresión:

```
# Ejemplo de declaración de cadenas de caracteres

cadena = 'Esto es una cadena de caracteres'
```

Para crear una cadena de caracteres utilizando triples comillas dobles, se utilizaría la siguiente expresión:

```
# Ejemplo de declaración de cadenas de caracteres

cadena = """

Esto es una cadena de caracteres

que ocupa varias líneas

"""
```

5.1.2 Inicialización (Python)

Una vez que se ha creado una cadena de caracteres, se puede inicializar o modificar utilizando la sintaxis de asignación. Por ejemplo, para inicializar una variable "mensaje" con la cadena "Hola, mundo!", se utilizaría la siguiente expresión:

```
# Ejemplo de Inicialización

mensaje = "Hola, mundo!"
```

También se puede modificar una cadena de caracteres existente utilizando la sintaxis de asignación y los operadores de concatenación (+) o multiplicación (*). Por ejemplo, para agregar el texto "Bienvenidos" al principio de la cadena "Hola, mundo!", se utilizaría la siguiente expresión:

```
# Ejemplo de Inicialización

mensaje = "Bienvenidos " + mensaje
```

5.1.3 Operaciones: Acceso a elementos, conversaciones y concatenación(Python)

Las cadenas de caracteres, también conocidas como "strings", son una secuencia de caracteres que se pueden utilizar para almacenar y manipular texto en Python.

- **Acceso a elementos:**

Los caracteres en una cadena se numeran desde 0 hasta la longitud de la cadena menos uno. Para acceder a un elemento de una cadena, se utiliza la sintaxis de corchetes, indicando la posición del elemento que se quiere obtener. Por ejemplo, para obtener el primer carácter de la cadena "hola", se utilizaría la siguiente expresión:

```
# Ejemplo de acceso

cadena = "hola"

primer_caracter = cadena[0]
```

- **Conversiones:**

Las cadenas se pueden convertir a otros tipos de datos, como enteros o flotantes, utilizando las funciones de conversión incorporadas de Python. Por ejemplo, para convertir la cadena "42" a un entero, se utilizaría la función "int":

```
# Ejemplo de conversiones

cadena = "42"

entero = int(cadena)
```

- **Concatenación:**

La concatenación de cadenas se refiere a la combinación de dos o más cadenas en una sola cadena. Esto se puede hacer utilizando el operador de suma (+) o utilizando la función de concatenación incorporada de Python, "join()". Por ejemplo, para concatenar las cadenas "hola" y "mundo" utilizando el operador de suma, se utilizaría la siguiente expresión:

```
# Ejemplo de concatenación
cadena1 = "hola"
cadena2 = "mundo"
concatenacion = cadena1 + cadena2
```

Más info

Este artículo de Real Python, que profundiza en las operaciones de slicing y concatenación de las cadenas de caracteres en Python: https://www.geeksforgeeks.org/python-string/

5.2 Declaración (Java)

En Java, las cadenas de caracteres se representan por el tipo de datos String. Para declarar una cadena de caracteres, primero debemos definir una variable que tenga este tipo de dato. Podemos hacerlo de la siguiente manera:

```
// Ejemplo de declaración de cadena de caracteres
String miCadena;
```

En esta línea de código, estamos declarando una variable llamada miCadena que tiene un tipo de dato String. Sin embargo, en este momento, miCadena aún no tiene un valor asignado. Para asignar un valor a esta variable, podemos utilizar el operador = y escribir la cadena de caracteres que deseamos asignar:

```
// Ejemplo de declaración de cadena de caracteres
miCadena = "Hola, mundo!";
```

Ahora, la variable miCadena tiene el valor de la cadena de caracteres "Hola, mundo!".
También podemos declarar y asignar un valor a una cadena de caracteres en la misma línea de código. Por ejemplo:

```
// Ejemplo de declaración de cadena de caracteres
String miOtraCadena = "¡Estoy aprendiendo Java!";
```

En esta línea de código, estamos declarando una variable llamada miOtraCadena y le estamos asignando el valor de la cadena de caracteres "¡ Estoy aprendiendo Java!" en la misma línea.
Es importante tener en cuenta que las cadenas de caracteres en Java son inmutables, lo que significa que no se pueden cambiar una vez que se han creado. Si queremos modificar una cadena de caracteres existente, debemos crear una nueva cadena de caracteres y asignarla a una nueva variable o a la misma variable, sobrescribiendo el valor anterior.

5.3 Creación de cadenas de caracteres (Java)

La creación de cadenas de caracteres en Java es un concepto importante que es necesario entender para poder trabajar con texto en este lenguaje de programación. Aquí te explico cómo se crean cadenas de caracteres en Java, con algunos ejemplos:

- **Crear una cadena de caracteres vacía:**

```
// Crear una cadena de caracteres vacía:

String cadenaVacia = "";
```

En este ejemplo, se crea una variable llamada cadenaVacia que tiene una cadena de caracteres vacía como valor. Esto puede ser útil si queremos crear una cadena de caracteres que aún no tiene un valor específico.

- **Crear una cadena de caracteres con un valor específico:**

```
// Crear una cadena de caracteres con un valor específico:

String saludo = "Hola, mundo!";
```

En este ejemplo, se crea una variable llamada saludo que tiene la cadena de caracteres "Hola, mundo!" como valor. Esto es útil cuando queremos trabajar con una cadena de caracteres específica.

- **Crear una cadena de caracteres utilizando concatenación:**

```
// Crear una cadena de caracteres utilizando concatenación:

String nombre = "Juan";

String mensaje = "Hola, " + nombre + "!";
```

En este ejemplo, se crean dos variables: nombre, que tiene el valor "Juan", y mensaje, que utiliza concatenación para crear la cadena de caracteres "Hola, Juan!". En la línea de código donde se crea la variable mensaje, se utiliza el

operador + para concatenar tres cadenas de caracteres diferentes: "Hola, ", el valor de la variable nombre y "!".

- **Crear una cadena de caracteres utilizando el constructor de la clase String**

```
// Crear una cadena de caracteres utilizando el constructor de la clase String:

char[] letras = {'H', 'o', 'l', 'a'};

String saludo = new String(letras);
```

En este ejemplo, se crea un arreglo de caracteres llamado letras que tiene los caracteres "H", "o", "l" y "a". Luego, se utiliza el constructor de la clase String para crear la cadena de caracteres "Hola" a partir de este arreglo.

5.4 Inicialización (Java)

La inicialización de cadenas de caracteres en Java es un tema fundamental para entender cómo trabajar con texto en este lenguaje de programación.
Para inicializar una cadena de caracteres en Java, se utiliza el tipo de datos String. Una cadena de caracteres es simplemente una serie de caracteres que se encuentran dentro de comillas dobles. Por ejemplo, la siguiente línea de código inicializa una cadena de caracteres con el texto "Hola Mundo":

```
// Ejemplo de Inicialización

String mensaje = "Hola Mundo";
```

En esta línea de código, la palabra String indica que se está declarando una variable de tipo cadena de caracteres. La variable se llama mensaje y se le asigna el valor "Hola Mundo" mediante el uso de las comillas dobles.
Es importante tener en cuenta que las cadenas de caracteres en Java son objetos, lo que significa que se pueden realizar diferentes operaciones con ellas. Por ejemplo, se pueden concatenar dos cadenas de caracteres utilizando el operador +:

```
// Ejemplo de Inicialización

String saludo = "Hola";

String nombre = "Juan";

String mensaje = saludo + " " + nombre;
```

En este ejemplo, se crean tres variables de tipo cadena de caracteres: saludo, nombre y mensaje. Las dos primeras contienen el texto "Hola" y "Juan", respectivamente. La variable mensaje se inicializa concatenando las variables “ saludo” y “ nombre” .

5.5 Operaciones: Acceso a elementos, conversaciones, concatenación (Java)

Las cadenas de caracteres, también conocidas como "strings", son una secuencia de caracteres que se pueden utilizar para almacenar y manipular texto en Java.

- **Acceso a elementos**

Los caracteres en una cadena se numeran desde 0 hasta la longitud de la cadena menos uno. Para acceder a un elemento de una cadena, se utiliza la sintaxis de corchetes, indicando la posición del elemento que se quiere obtener. Por ejemplo, para obtener el primer carácter de la cadena "hola", se utilizaría la siguiente expresión:

```
// Acceso a elementos:
String cadena = "hola";
char primerCaracter = cadena.charAt(0);
```

- **Conversiones**

Las cadenas se pueden convertir a otros tipos de datos, como enteros o flotantes, utilizando las funciones de conversión incorporadas de Java. Por ejemplo, para convertir la cadena "42" a un entero, se utilizaría el método "parseInt" de la clase Integer:

```
// Ejemplo de conversiones
String cadena = "42";
int entero = Integer.parseInt(cadena);
```

- **Concatenación**

La concatenación de cadenas se refiere a la combinación de dos o más cadenas en una sola cadena. Esto se puede hacer utilizando el operador de suma (+) o utilizando el método de concatenación incorporado de Java, "concat()". Por ejemplo, para concatenar las cadenas "hola" y "mundo" utilizando el operador de suma, se utilizaría la siguiente expresión:

```
// Ejemplo de concatenación
String cadena1 = "hola";
```

```
String cadena2 = "mundo";
String concatenacion = cadena1 + cadena2;
```

O utilizando el método "concat()":

```
// Ejemplo de concatenación
String cadena1 = "hola";
String cadena2 = "mundo";
String concatenacion = cadena1.concat(cadena2);
```

6. Resumen

En esta unidad hemos aprendido fundamentalmente cómo trabajar con estructuras de almacenamiento de datos en programación. Se cubren los siguientes temas:
Arrays unidimensionales y multidimensionales: se aprende a declarar, crear e inicializar arrays, a acceder a sus elementos, recorrerlos, buscar y ordenar sus elementos. Además, se enseña a implementar algoritmos que utilicen estas estructuras de datos.

Cadenas de caracteres: se aprende a declarar, crear e inicializar cadenas de caracteres, y a realizar operaciones como acceder a sus elementos, conversiones y concatenación.

En los Arrays unidimensionales y multidimensionales, se ha conocido la declaración, la creación de arrays unidimensionales y multidimensionales, la inicialización y el acceso a elementos, entre otros conceptos.
En general, estos conceptos son fundamentales para trabajar con estructuras de almacenamiento de datos en cualquier lenguaje de programación. Es importante entender cómo se declaran, crean, inicializan y manipulan los elementos de los arrays, así como cómo se realizan búsquedas y ordenaciones para optimizar el rendimiento del programa.
Las cadenas de caracteres son una estructura de datos importante en la programación y se utilizan ampliamente en la manipulación de texto.
Es importante tener un buen manejo de las cadenas de caracteres ya que son ampliamente utilizadas en la programación y se presentan en la mayoría de los problemas de programación.

Glosario

- **Declaración:** proceso de informar al compilador sobre la existencia de un array y las propiedades de este, como su tamaño y tipo de datos.

- **Creación:** proceso de asignar memoria para almacenar los elementos del array.

- **Inicialización:** proceso de asignar valores a los elementos del array durante la creación o en un momento posterior.

- **Operaciones:** procesos que se pueden realizar en una cadena, como la comparación, la concatenación o la conversión de mayúsculas a minúsculas.

- **Conversiones:** proceso de cambiar el formato o tipo de datos de una cadena, como convertir una cadena en un entero.

- **Array:** estructura de datos que almacena una colección de elementos del mismo tipo y se accede a cada elemento mediante un índice.

- **Subcadena:** parte de una cadena de caracteres que se puede extraer utilizando un índice de inicio y un índice de finalización.

Bibliografía

Matthes, E. (2019). *Python Crash Course, 2nd Edition: A Hands-On, Project-Based Introduction to Programming* (2.ª ed.). No Starch Press.

Automate the Boring Stuff with Python. (s. f.). https://automatetheboringstuff.com/#toc

Schildt, H. (2018). *Java: A Beginner's Guide* (8.ª ed.). McGraw Hill.

Goodrich; Tamassia;, Michael. T. R. M. H. ; G. (2014). *Data Structures and Algorithms in Java* (6.ª ed.). Wiley.

Tech Hub <FP> euroformac